

AI DOCUMENT INTELLIGENCE PLATFORM

AI-powered document understanding, retrieval and grounded question answering • Rinoy & Sreelakshmi

WHAT IT DOES	Users upload PDF documents and ask natural-language questions. The platform extracts and chunks the documents, creates embeddings, retrieves the most relevant passages, and sends only that evidence to Google Gemini to generate a grounded answer with traceable sources.
CORE GOAL	Turn a collection of difficult-to-search documents into searchable knowledge while reducing manual PDF lookup and avoiding answers based solely on the model's general knowledge.

SYSTEM ARCHITECTURE & TECHNOLOGY STACK

FRONTEND React • TypeScript • React Router Login • Register • Dashboard • Documents • Query • History • Admin Dashboard	API / APPLICATION ASP.NET Core • .NET 8 • REST APIs DTOs • entities • service interfaces • controllers • dependency injection	INFRASTRUCTURE EF Core • SQL Server • ASP.NET Identity • JWT File storage • background processing • RAG • Gemini • PdfPig
---	---	---

React Frontend UI + route protection	→	ASP.NET Core API HTTP + authentication	→	Application Layer contracts + domain structures	→	Infrastructure DB + storage + AI
---	---	---	---	--	---	-------------------------------------

1 DOCUMENT INGESTION Upload → Validate → Store → DB record → Queue → Process PDF text is extracted with PdfPig . Text is divided into chunks carrying page number, chunk index and token count. Each chunk is converted into an embedding and stored with its chunk record. Asynchronous processing: heavy extraction, chunking and embedding run through a background queue so the upload request stays responsive. Document status is tracked as Uploaded / Processing / Ready / Failed . 2 RETRIEVAL At query time, the question is embedded using the same embedding approach. The service compares the query vector with stored vectors using cosine similarity , sorts by relevance and returns the requested Top-K chunks. Current storage design: embeddings are stored as JSON in SQL Server rather than in a dedicated vector database.	3 RAG GENERATION Question → Query Embedding → Similarity Retrieval → Top-K Chunks → Grounded Prompt → Gemini → Answer + Sources The retrieved chunks become the document context for Gemini. The prompt explicitly instructs the model to use only the supplied context , avoid inventing facts or using outside knowledge, and state when the information is unavailable. Key principle: retrieval supplies the knowledge; Gemini supplies the natural-language generation. 4 TRACEABILITY Each query records the user, query, answer, grounded status, created time, response time, source chunks and relevance scores . This creates an audit trail connecting an answer back to the exact document passages used to produce it.
--	--

AUTHENTICATION & AUTHORIZATION Credentials are validated at login and a JWT is issued. The frontend sends the bearer token on API calls. ASP.NET Core reads claims and enforces roles. Admin controllers use [Authorize(Roles = "Admin")] ; frontend route guards are UX protection, while backend authorization is the real security boundary.	ADMIN FUNCTIONALITY Administrators can list/search/delete users; list/search/get/delete documents; upload documents; query across documents; and view/search all query history with filtering and source details. Admin query retrieval deliberately runs without a user filter.	USER FUNCTIONALITY Document users can register/login, upload documents, ask questions, view grounded answers and sources, and inspect their own query history through the frontend.
--	--	---

SECURITY HARDENING — DOCUMENT ISOLATION	The retrieval path was hardened so a normal user's query carries the authenticated UserId through QueryController → RagService → RetrievalRequest → RetrievalService. Retrieval joins chunks to Documents and filters by UploadedByUserId . This prevents another user's document chunks from entering normal-user retrieval, while UserId = null preserves intentional admin-wide retrieval.
--	--

DATA MODEL User → Document → DocumentChunk → Embedding User → QueryHistory → QueryHistorySource → DocumentChunk Chunk-level embeddings allow retrieval to pinpoint relevant passages instead of representing only whole documents.	ADMIN API SURFACE Users: GET, DELETE /admin/users Documents: GET, POST, DELETE /admin/documents Query: POST /admin/query History: GET /admin/query-history Admin endpoints require the Admin role.	FRONTEND SECURITY AuthContext maintains the current authenticated user. ProtectedRoute guards authenticated pages. AdminRoute checks authentication and then the Admin role before showing the admin dashboard. The browser never connects directly to SQL Server, file storage or Gemini.
--	--	--

TESTING & ENGINEERING VALIDATION

Automated tests: unit, controller, service and integration coverage across authentication, controllers, document services, admin functionality, retrieval/RAG, validation, file storage, error handling and query history.	Retrieval/RAG cases: empty/invalid input, no embeddings, similarity ranking, Top-K limits, chunk metadata, invalid JSON vectors, mismatched dimensions, zero vectors, and RAG response/error scenarios.	Security regression: tests verify User A receives User A's documents, cannot receive User B's documents, while global/admin retrieval can retrieve documents belonging to multiple users. Swagger was also used for manual API verification.
---	--	---

DEVELOPMENT / DEMO FLOW Architecture → Authentication → Documents → Processing → Embeddings → Retrieval → RAG → History → Frontend → Admin → Hardening	FUTURE IMPROVEMENTS • Allow admins to view documents directly. • Allow admins to grant users access to other users' documents. • Display the document name associated with the source used to answer a query. • Support additional document types beyond PDF.
--	--

END-TO-END SYSTEM: Login → Upload → Validate → Process → Embed → Retrieve → Generate → Answer + Sources → Query History → Admin Management